

Universidad Autónoma Metropolitana

Licenciatura en Tecnología y Sistemas de Información

Enjambre Abismal

**Simulación interactiva de comportamiento colectivo
con agentes autónomos en Three.js**

Proyecto Final

Integrantes:

Carlos Ricardo Pérez Richards

Matrícula: 2223067956

Iker Adan Gonzalez Vega

Matrícula: 2223067830

Vicente García Alfaro

Matrícula: 2213062190

14 de julio de 2026

Índice

1. Resumen	3
2. Introducción	3
3. Objetivo del Proyecto	4
4. Estructura General del Proyecto	4
4.1. index.html	4
4.2. style.css	5
4.3. World.js	5
4.4. Agent.js	5
4.5. main.js	6
5. Escena Base	6
5.1. Renderer	6
5.2. Cámara	6
5.3. Controles Orbitales	6
5.4. Iluminación	6
6. Entorno Abismal	7
6.1. Fondo y Niebla	7
6.2. Suelo Procedural	7
6.3. Partículas Bioluminiscentes	7
6.4. Rocas y Columnas de Luz	7
7. Diseño del Agente	8
7.1. Estructura Visual	8
7.2. Geometrías	8
7.3. Materiales	9
8. Modelo de Movimiento	9
8.1. Estado del Agente	9
8.2. Actualización	10
9. Implementación del Algoritmo Boids	10
9.1. Separación	10
9.2. Alineación	11
9.3. Cohesión	11
9.4. Combinación de Fuerzas	11

9.5. Pseudocódigo	12
10. Animación Procedural	12
10.1. Pulsación de la Campana	12
10.2. Movimiento de Tentáculos	12
11. Shader Personalizado	13
11.1. Motivación	13
11.2. Uniforme de Tiempo	13
11.3. Deformación	13
11.4. Ventaja Técnica	14
12. Optimización del Renderizado	14
12.1. Geometrías Compartidas	14
12.2. Materiales Compartidos	14
12.3. Eliminación de Luces Locales	14
12.4. Decisión sobre Instancing	14
13. Interfaz de Usuario	15
13.1. Elementos del HUD	15
13.2. Controles	15
14. Resultados Técnicos	16
15. Análisis Técnico	16
16. Conclusión	17

1. Resumen

El proyecto *Enjambre Abismal* es una simulación interactiva en tiempo real de un sistema de enjambre dentro de un ambiente submarino abismal. El sistema representa 205 agentes autónomos con forma de medusas, los cuales se desplazan en un espacio tridimensional utilizando reglas de comportamiento colectivo.

El comportamiento principal se basa en el algoritmo Boids, compuesto por tres reglas: separación, alineación y cohesión. Cada agente analiza la posición y velocidad de sus vecinos dentro de un radio de percepción y, con base en ello, ajusta su movimiento. La combinación de estas reglas permite que el grupo forme patrones de movimiento orgánicos sin necesidad de definir una trayectoria global para todo el enjambre.

El proyecto fue desarrollado en Three.js e integra varios componentes técnicos trabajados durante el curso: escena base 3D, cámara perspectiva, luces, controles orbitales, entorno visual, agentes articulados, animación procedural, shaders personalizados y optimización de rendimiento. La escena incluye niebla, partículas bioluminiscentes, suelo procedural, rocas decorativas y una interfaz de usuario para controlar la simulación durante la demostración.

2. Introducción

La propuesta inicial del proyecto consistía en construir un sistema de agentes acuáticos que simularan el comportamiento colectivo de un cardumen. La idea central era representar cómo múltiples entidades simples pueden generar un movimiento organizado a partir de reglas locales.

Durante el desarrollo, el concepto visual evolucionó hacia un enjambre de medusas abisemales. Esta decisión permitió mantener el contexto acuático original, pero agregó mayor riqueza visual y técnica. A diferencia de un pez simple, una medusa permite representar un cuerpo translúcido, tentáculos articulados, movimiento pulsante y efectos de deformación mediante shader.

El proyecto final no se limita a colocar varios objetos en movimiento. Cada agente contiene una estructura interna, una lógica de comportamiento, animación independiente y una deformación visual aplicada sobre su cuerpo. A nivel global, todos los agentes interactúan mediante reglas de flocking, lo que produce una simulación colectiva dinámica.

La documentación tiene como finalidad explicar la estructura técnica del proyecto para facilitar su presentación. Por ello, se describen los módulos principales, la escena base, el diseño del agente, el algoritmo Boids, el shader utilizado, las optimizaciones realizadas y la interfaz de usuario.

3. Objetivo del Proyecto

El objetivo general fue construir una simulación completa de enjambre que integrara los contenidos principales del curso en una sola aplicación gráfica interactiva.

Los objetivos específicos fueron:

- Crear una escena tridimensional con estética submarina abismal.
- Representar más de 200 agentes simultáneos en tiempo real.
- Implementar comportamiento colectivo mediante separación, alineación y cohesión.
- Diseñar un agente articulado con animación independiente.
- Incorporar un shader propio de deformación de vértices.
- Mantener un rendimiento aceptable mediante decisiones de optimización.
- Agregar una interfaz de usuario para controlar aspectos básicos de la simulación.

Estos objetivos permiten demostrar la integración entre programación gráfica, simulación de agentes, animación y diseño visual.

4. Estructura General del Proyecto

El proyecto se organiza en cinco archivos principales. Cada uno tiene una responsabilidad específica dentro del sistema.

```
Proyecto/  
  index.html  
  main.js  
  Agent.js  
  World.js  
  style.css
```

4.1. index.html

El archivo `index.html` define la estructura base de la página. En él se encuentra el HUD de la simulación, que muestra el título del proyecto, el número de agentes, el contador de FPS y los controles principales.

También se incluye el `importmap` utilizado para cargar Three.js desde CDN, así como la referencia al archivo `main.js`, que funciona como punto de entrada de la aplicación.

4.2. `style.css`

El archivo `style.css` define la apariencia visual de la interfaz. El diseño utiliza una estética minimalista con colores azulados, tipografía monoespaciada y elementos discretos. Esto mantiene la coherencia con el concepto de un entorno abismal y evita que la interfaz compita visualmente con la simulación.

El HUD se posiciona sobre el canvas. La mayoría de sus elementos no bloquean la interacción con la escena, excepto los controles que requieren eventos del mouse.

4.3. `World.js`

El archivo `World.js` contiene la clase `World`. Esta clase encapsula la configuración general de la escena 3D. Sus responsabilidades principales son:

- Crear el renderer.
- Inicializar la escena.
- Configurar la cámara perspectiva.
- Activar los controles orbitales.
- Crear el sistema de luces.
- Construir el entorno abismal.
- Manejar el cambio de tamaño de la ventana.
- Ejecutar el ciclo principal de renderizado.

Separar esta lógica en una clase permite que el resto del proyecto se concentre en el comportamiento de los agentes.

4.4. `Agent.js`

El archivo `Agent.js` define la clase `Agent`. Esta clase representa a cada medusa del enjambre. En ella se concentran tres partes fundamentales:

- La construcción visual del agente.
- La lógica de comportamiento Boids.
- La animación procedural y el shader de deformación.

Cada agente mantiene su propia posición, velocidad, aceleración, escala, fase de animación, límites espaciales y pesos de comportamiento.

4.5. `main.js`

El archivo `main.js` funciona como coordinador principal del proyecto. Primero crea el mundo base, después define los parámetros del enjambre y finalmente genera los 205 agentes.

También conecta los controles del HUD, como el slider de velocidad, el botón de pausa, el botón para reiniciar la cámara y el botón para dispersar a los agentes. En el loop principal, `main.js` aplica el comportamiento Boids y actualiza el movimiento de cada agente.

5. Escena Base

La escena base fue construida con `Three.js`. Su función es establecer el espacio donde se desarrolla la simulación. Esta escena proporciona cámara, iluminación, controles, entorno visual y ciclo de renderizado.

5.1. `Renderer`

El renderer utilizado es `WebGLRenderer`. Se configuró con `antialiasing` para suavizar los bordes, control de pixel ratio para ajustar el consumo gráfico, sombras suaves y salida de color sRGB. También se activó el tone mapping ACES, lo que ayuda a conseguir una imagen con mejor contraste y apariencia visual más consistente.

Esta configuración es importante porque la escena utiliza colores oscuros, luces azuladas y materiales transparentes. Sin una configuración adecuada, los agentes podrían perder legibilidad dentro del fondo abismal.

5.2. Cámara

La cámara principal es una `PerspectiveCamera`. Su campo de visión permite observar tanto el comportamiento global del enjambre como acercarse a los agentes. La cámara se coloca inicialmente alejada del centro para que el usuario pueda ver la distribución general de las medusas.

5.3. Controles Orbitales

Se utilizan `OrbitControls` para permitir la exploración de la escena. El usuario puede rotar la cámara, acercarse, alejarse y recuperar una vista general. El damping agrega una transición suave al movimiento de cámara, lo que mejora la presentación durante la demo.

5.4. Iluminación

La escena utiliza una combinación de luces globales:

- Luz hemisférica para simular iluminación ambiental submarina.
- Luz direccional principal con sombras.
- Luz de relleno lateral.
- Luz de borde para reforzar profundidad y silueta.

Esta iluminación permite que las medusas se vean integradas en un entorno oscuro, pero sin desaparecer visualmente.

6. Entorno Abismal

El entorno se diseñó para comunicar una escena submarina profunda. No se trata de un espacio vacío, sino de un ambiente construido con varios elementos visuales.

6.1. Fondo y Niebla

La escena utiliza un color de fondo oscuro y niebla exponencial. La niebla cumple dos funciones: aporta profundidad visual y suaviza la aparición de objetos lejanos. En un enjambre con muchos agentes, esto ayuda a que la composición sea más legible.

6.2. Suelo Procedural

El suelo marino se genera mediante una textura procedural creada en un canvas. Esta textura se repite sobre un plano grande y simula variaciones visuales del fondo abismal. Al ser procedural, no depende de imágenes externas.

6.3. Partículas Bioluminiscentes

El entorno incluye 1200 partículas distribuidas en el espacio. Estas partículas usan colores entre azul tenue y cian, con transparencia y atenuación por distancia. Su movimiento rotacional lento genera una sensación de suspensión acuática.

6.4. Rocas y Columnas de Luz

También se agregan rocas de baja poligonización y columnas de luz decorativas. Estos elementos ayudan a dar escala y contexto a la escena, sin aumentar demasiado la complejidad geométrica.

7. Diseño del Agente

Cada agente representa una medusa construida de forma procedural. Esta decisión permite explicar claramente cómo está compuesto el modelo y cómo se anima.

7.1. Estructura Visual

Cada medusa tiene una raíz principal representada por un `THREE.Group`. Dentro de esa raíz se agregan las partes visuales del cuerpo:

- Campana principal.
- Aro inferior.
- Seis tentáculos.
- Tres segmentos por tentáculo.
- Pequeños nodos luminosos en las puntas.

La estructura puede entenderse de la siguiente manera:

```
Agent
root
  bellGroup
    bellMesh
    rimMesh
    tentacle 1
      baseGroup
      midGroup
      tipGroup
    tentacle 2
    ...
```

Esta jerarquía permite que cada parte se anime de forma local, pero manteniendo relación con el cuerpo completo.

7.2. Geometrías

Las geometrías principales se crean una sola vez y se comparten entre agentes. Entre ellas se encuentran:

- `SphereGeometry` para la campana.

- `TorusGeometry` para el aro inferior.
- `CylinderGeometry` para los segmentos de los tentáculos.
- `SphereGeometry` pequeña para los nodos de las puntas.

Usar geometrías simples permite mantener una buena relación entre calidad visual y rendimiento.

7.3. Materiales

Los materiales principales son `MeshPhongMaterial`. Estos materiales permiten transparencia, emisión ligera y brillo especular. La transparencia ayuda a que la medusa tenga apariencia orgánica y acuática.

Los materiales base se comparten entre agentes. Cuando se necesita color individual, se clona el material de la campana y se modifica su color y emisión. Esto permite variar la apariencia del enjambre sin duplicar todos los recursos.

8. Modelo de Movimiento

Cada agente se mueve usando un modelo simple de física basado en posición, velocidad y aceleración.

8.1. Estado del Agente

Cada agente mantiene:

- Posición actual.
- Velocidad.
- Aceleración.
- Velocidad máxima.
- Fuerza máxima.
- Radio de percepción.
- Pesos de separación, alineación y cohesión.

La aceleración acumula las fuerzas generadas por las reglas Boids. Después, esta aceleración modifica la velocidad. Finalmente, la velocidad actualiza la posición.

8.2. Actualización

En cada frame, el agente realiza el siguiente proceso:

1. Recibe fuerzas externas generadas por el comportamiento Boids.
2. Suma la aceleración a la velocidad.
3. Limita la velocidad con `maxSpeed`.
4. Actualiza la posición usando el delta de tiempo.
5. Limpia la aceleración.
6. Ajusta la orientación para apuntar hacia la dirección de movimiento.
7. Ejecuta la animación local de campana y tentáculos.
8. Verifica los límites espaciales.

Este proceso produce un movimiento continuo y evita que los agentes aceleren indefinidamente.

9. Implementación del Algoritmo Boids

El comportamiento colectivo del enjambre se basa en tres reglas locales: separación, alineación y cohesión. Cada regla se calcula comparando un agente con los demás agentes dentro de su radio de percepción.

9.1. Separación

La separación evita que los agentes se amontonen. Cuando otro agente se encuentra dentro del radio de percepción, se calcula un vector que apunta en dirección opuesta al vecino. La fuerza es mayor cuando la distancia es menor.

$$F_{sep} = \sum_{i=1}^n \frac{p - p_i}{||p - p_i||}$$

Donde p es la posición del agente actual y p_i la posición de cada vecino.

9.2. Alineación

La alineación permite que un agente ajuste su dirección con base en la velocidad promedio de sus vecinos. Esta regla genera la sensación de que el grupo avanza de manera coordinada.

$$F_{ali} = \frac{1}{n} \sum_{i=1}^n v_i$$

Donde v_i representa la velocidad de cada vecino.

9.3. Cohesión

La cohesión atrae al agente hacia el centro promedio de sus vecinos. Esta regla mantiene unido al grupo y evita que los agentes se dispersen completamente.

$$C = \frac{1}{n} \sum_{i=1}^n p_i$$

$$F_{coh} = C - p$$

9.4. Combinación de Fuerzas

Las tres reglas se combinan mediante pesos:

$$F = w_{sep}F_{sep} + w_{ali}F_{ali} + w_{coh}F_{coh}$$

En el proyecto final se definieron los siguientes valores base dentro de la configuración del enjambre:

Parámetro	Valor
Peso de separación	1.5
Peso de alineación	1.0
Peso de cohesión	1.0
Radio de percepción	3.5
Velocidad máxima	2.0
Fuerza máxima	0.3

Cuadro 1: Configuración base del comportamiento Boids.

Estos valores permiten que el enjambre mantenga una distribución estable: los agentes evitan estar demasiado juntos, se orientan hacia el movimiento promedio del grupo y tienden a mantenerse agrupados.

9.5. Pseudocódigo

```
para cada agente:
    separacion = calcularSeparacion(agentes)
    alineacion = calcularAlineacion(agentes)
    cohesion = calcularCohesion(agentes)

    fuerzaTotal =
        separacion * w_sep +
        alineacion * w_ali +
        cohesion * w_coh

    aplicarFuerza(fuerzaTotal)

para cada agente:
    actualizarMovimiento(deltaTime)
```

En la simulación, este proceso se repite en cada frame. Por eso el movimiento no es pregrabado, sino calculado en tiempo real.

10. Animación Procedural

Además del movimiento general producido por Boids, cada medusa tiene animaciones internas. Estas animaciones están basadas en funciones trigonométricas y en una fase individual.

10.1. Pulsación de la Campana

La campana cambia de escala para simular una contracción orgánica. La expresión principal es:

$$pulse = 1 + \sin(t \cdot 2,4) \cdot 0,09$$

Este valor se aplica al grupo de la campana y genera un movimiento de expansión y contracción.

10.2. Movimiento de Tentáculos

Cada tentáculo tiene una fase distinta. Además, sus segmentos base, medio y punta se mueven con diferentes frecuencias y amplitudes.

```
baseGroup.rotation.x = sin(t * 1.8 + phase) * 0.3
midGroup.rotation.x  = sin(t * 2.2 + phase + 0.8) * 0.5
tipGroup.rotation.x  = sin(t * 2.8 + phase + 1.6) * 0.7
```

Esto evita que todos los tentáculos se muevan igual. La diferencia de fase hace que el movimiento se vea más natural y menos mecánico.

11. Shader Personalizado

El proyecto incluye un shader propio aplicado sobre el material de la campana. Para lograrlo se utilizó `onBeforeCompile`, una técnica que permite modificar el shader interno de un material de Three.js antes de que se compile.

11.1. Motivación

La intención del shader fue agregar una deformación orgánica a la campana sin modificar los vértices desde CPU. La deformación se calcula directamente en el vertex shader, por lo que ocurre en GPU.

11.2. Uniforme de Tiempo

El shader agrega un uniforme llamado `uTime`. Este valor cambia continuamente y permite animar la deformación.

```
uniform float uTime;
```

11.3. Deformación

La deformación se calcula con una función seno:

```
float oscilacion =
    sin(position.y * 5.0 + uTime * 3.0) * 0.15;

transformed.x += oscilacion;
transformed.z += oscilacion;
```

La oscilación depende de la altura local del vértice y del tiempo. Esto provoca que la campana tenga un movimiento lateral suave, similar a una vibración orgánica.

11.4. Ventaja Técnica

La ventaja de esta solución es que el proyecto conserva las propiedades del `MeshPhongMaterial`, como iluminación difusa, especularidad, transparencia y emisión. En lugar de escribir un shader completo desde cero, se modifica únicamente la parte necesaria del vertex shader.

12. Optimización del Renderizado

Uno de los principales retos fue representar 205 agentes en tiempo real. Cada medusa está formada por múltiples mallas, por lo que el costo de renderizado puede crecer rápidamente. Para mantener el rendimiento, se aplicaron varias decisiones de optimización.

12.1. Geometrías Compartidas

Las geometrías se crean una sola vez y se reutilizan. Esto reduce el uso de memoria y evita repetir buffers para cada agente.

12.2. Materiales Compartidos

Los materiales base también se comparten. Esto reduce la cantidad de objetos de material en memoria y simplifica el renderizado. Solo se clona el material de la campana cuando se requiere aplicar un color distinto.

12.3. Eliminación de Luces Locales

Se decidió eliminar luces puntuales individuales en cada medusa. Aunque estas luces podían reforzar el brillo visual, habrían incrementado considerablemente el costo de iluminación. En su lugar, se utilizaron luces globales y materiales con emisión ligera.

Esta decisión permitió conservar una apariencia luminosa sin multiplicar el número de fuentes de luz dinámicas.

12.4. Decisión sobre Instancing

Durante el curso se trabajó con técnicas como `InstancedMesh` y VAT. Sin embargo, para la versión final se priorizó conservar la estructura articulada de las medusas. Usar instancing completo habría reducido draw calls, pero también habría limitado el control individual sobre los tentáculos y las animaciones jerárquicas.

Por esta razón, el proyecto utiliza una estrategia de optimización selectiva: geometrías compartidas, materiales compartidos, eliminación de luces locales y animaciones matemáticas ligeras.

13. Interfaz de Usuario

El proyecto incluye una interfaz simple para controlar la simulación durante la demostración.

13.1. Elementos del HUD

El HUD muestra:

- Nombre del proyecto.
- Cantidad de agentes.
- Contador de FPS.
- Slider de velocidad.
- Botón de pausa.
- Botón de cámara.
- Botón de dispersión.

13.2. Controles

Control	Función
Slider de velocidad	Modifica el multiplicador global de tiempo
Pausar	Detiene la actualización física de los agentes
Cámara	Regresa la cámara a una vista inicial
Dispersar	Reubica los agentes en posiciones aleatorias
Mouse drag	Rota la cámara orbital
Scroll	Acerca o aleja la cámara

Cuadro 2: Controles principales de la simulación.

La función de dispersar es útil para mostrar el comportamiento emergente, ya que rompe temporalmente la agrupación y permite observar cómo el enjambre vuelve a reorganizarse mediante las reglas de Boids.

14. Resultados Técnicos

El sistema final representa 205 medusas autónomas dentro de una escena abismal. Cada agente tiene posición inicial, rotación, escala, color y fase de animación distintas. Esto evita que el enjambre se vea uniforme y ayuda a construir una simulación más orgánica.

Elemento	Resultado
Motor gráfico	Three.js
Cantidad de agentes	205
Tipo de agente	Medusa articulada
Reglas implementadas	Separación, alineación y cohesión
Entorno	Submarino abismal
Shader propio	Deformación de vértices en campana
Partículas ambientales	1200
Control de cámara	OrbitControls
Interfaz	HUD interactivo
Optimización principal	Recursos compartidos y eliminación de luces locales

Cuadro 3: Resumen técnico del sistema final.

15. Análisis Técnico

El proyecto se puede entender como la integración de tres capas: capa visual, capa de comportamiento y capa de interacción.

La capa visual está formada por el entorno abismal, los agentes, las luces, la niebla, las partículas y los materiales. Esta capa define la apariencia del sistema y permite que el comportamiento del enjambre se observe de forma clara.

La capa de comportamiento corresponde al modelo Boids. Cada agente calcula separación, alineación y cohesión con base en el resto del grupo. Aunque estas reglas son simples, al repetirse en todos los agentes producen un movimiento colectivo complejo.

La capa de interacción está compuesta por el HUD y los controles de cámara. Esta capa permite modificar la velocidad, pausar la simulación, reiniciar la cámara y dispersar los agentes para observar cómo se reorganiza el enjambre.

El principal reto técnico fue equilibrar la cantidad de agentes con la complejidad visual de cada uno. Una medusa articulada tiene más costo que una esfera o un triángulo simple. Sin embargo, mantener esa estructura permitió mostrar animación jerárquica, movimiento orgánico y un diseño visual más atractivo. Por ello, la optimización no consistió en simplificar todo el sistema, sino en seleccionar qué costos reducir sin perder la intención visual.

16. Conclusión

Enjambre Abismal integra los elementos principales desarrollados durante el curso en una simulación interactiva. El proyecto combina escena 3D, agentes autónomos, flocking, animación procedural, shader personalizado, optimización de recursos e interfaz de usuario.

El comportamiento del enjambre surge de reglas locales aplicadas por cada agente. La separación evita aglomeraciones, la alineación coordina direcciones y la cohesión mantiene unido al grupo. La suma de estas reglas genera un movimiento emergente visible dentro de la escena.

La elección de medusas como agentes permitió enriquecer la presentación visual y aplicar técnicas de animación jerárquica. Además, el shader de deformación de vértices refuerza la sensación orgánica del movimiento sin requerir geometría adicional ni animación manual desde CPU.

En conjunto, el proyecto demuestra cómo los conceptos de gráficos 3D, simulación de movimiento, shaders y sistemas de enjambre pueden integrarse en una aplicación funcional y explicable técnicamente.